Module 9.2 - Template-Driven Form Validation

> **Learning Objectives:**
>
> - Understand form states (valid, invalid, touched, dirty, pristine)
> - Implement built-in Angular validators (required, minlength, maxlength, pattern)
> - Differentiate HTML5 validation from Angular validation
> - Display validation error messages conditionally
> - Create custom validators with ngModel
> - Use validation states to control form behavior

Table of Contents

1. Understanding Form States

1.1 What are Form States?

Angular tracks the state of each form control and the form itself. These states help determine when to show errors, enable/disable buttons, and provide user feedback.

Key Form States:

State	Description	Opposite State
`valid`	All validation rules pass	`invalid`
`invalid`	One or more validation rules fail	`valid`
`pristine`	User has not changed the value	`dirty`
`dirty`	User has changed the value	`pristine`
`touched`	User focused then left the field	`untouched`
`untouched`	User has not focused the field	`touched`

1.2 Accessing Form States

Template:

```
<form #myForm="ngForm">
  <input
    type="text"
    name="username"
    [(ngModel)]="username"
    #usernameField="ngModel"
    required>

  <!-- Access form states -->
  <p>Form Valid: {{ myForm.valid }}</p>
  <p>Field Touched: {{ usernameField.touched }}</p>
  <p>Field Dirty: {{ usernameField.dirty }}</p>
</form>
```

Output:

Initial: Form Valid: false, Field Touched: false, Field Dirty: false
After typing: Form Valid: true, Field Touched: true, Field Dirty: true

1.3 State Transition Flow

Pristine → Dirty: Happens when user types in the field

Untouched → Touched: Happens when user focuses then leaves the field (blur event)

Invalid → Valid: Happens when all validation rules are satisfied

> **Key Point:** States are independent. A field can be `dirty` and `untouched` if programmatically changed, or `touched` and `pristine` if user focused without typing.

---

2. Built-in Angular Validators

2.1 Required Validator

The `required` validator ensures that a field must have a value.

Template:

```
<input
  name="fullName"
  [(ngModel)]="fullName"
  #nameField="ngModel"
  required>

<div *ngIf="nameField.invalid && nameField.touched" style="color: red;">
  Name is required!
</div>
```

Output:

Empty field (touched): Name is required! ← Error shown
With value: ← No error

2.2 Minlength Validator

The `minlength` validator ensures minimum character count.

Template:

```
<input
  type="password"
  name="password"
  [(ngModel)]="password"
  #passwordField="ngModel"
  required
  minlength="6">

<div *ngIf="passwordField.errors?.['minlength'] && passwordField.touched"
     style="color: red;">
  Password must be at least 6 characters
</div>
```

Output:

Input "pass": Error shown (too short)

Input "pass123": Valid ✓

2.3 Maxlength Validator

The `maxlength` validator limits maximum character count.

Template:

```
<input
  name="username"
  [(ngModel)]="username"
  maxlength="15">

<small>{{ username?.length || 0 }} / 15 characters</small>
```

> **Note:** `maxlength` is enforced by the browser - users cannot type beyond the limit.

2.4 Pattern Validator

The `pattern` validator uses regular expressions to validate input format.

Example - Email:

```
<input
  name="email"
  [(ngModel)]="email"
  #emailField="ngModel"
  pattern="^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$">

<div *ngIf="emailField.errors?.['pattern'] && emailField.touched"
     style="color: red;">
  Please enter a valid email address
</div>
```

Output:

Input: "john" → Invalid ❌

Input: "john@example.com" → Valid ✓

2.5 Common Pattern Examples

Pattern	Regular Expression	Description
Email	`^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$`	Valid email format
Phone (10 digits)	`^[0-9]{10}$`	Exactly 10 numbers
ZIP Code	`^[0-9]{5}(-[0-9]{4})?$`	5 digits or 5+4 format
Alphanumeric	`^[a-zA-Z0-9]+$`	Only letters and numbers
URL	`^https?://[^\s]+$`	Valid HTTP/HTTPS URL
Strong Password	`^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).{8,}$`	8+ chars, uppercase, lowercase, digit

3. HTML5 vs Angular Validation

3.1 Understanding the Difference

HTML5 Validation:

- Native browser validation
- Uses attributes like `required`, `min`, `max`, `pattern`
- Shows browser-default error messages
- Triggered on form submission
- Cannot be customized easily

Angular Validation:

- Framework-level validation
- Tracks form states programmatically
- Allows custom error messages
- Real-time validation feedback
- Full control over UI/UX

3.2 Disabling HTML5 Validation

When using Angular forms, you typically disable HTML5 validation to have full control over the validation experience.

Template:

```
<!-- With HTML5 Validation (default) -->
<form #myForm="ngForm">
  <input type="email" name="email" [(ngModel)]="email" required>
  <button type="submit">Submit</button>
</form>

<!-- Without HTML5 Validation (recommended for Angular) -->
<form #myForm="ngForm" novalidate>
```

```
  <input type="email" name="email" [(ngModel)]="email" required>
  <button type="submit">Submit</button>
</form>
```

Output:

With HTML5 validation (no novalidate):
[Submit clicked with empty email]
Browser shows: "Please fill out this field" ← Native browser popup

With novalidate attribute:
[Submit clicked with empty email]
No browser popup. Angular validation takes over.
You control when and how errors display.

3.3 Practical Example

Template with Angular Validation:

```
<form #myForm="ngForm" novalidate>
  <input
    type="email"
    name="email"
    [(ngModel)]="email"
    #emailField="ngModel"
    required
    pattern="^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$">

  <div *ngIf="emailField.invalid && emailField.touched" style="color: red;">
    <div *ngIf="emailField.errors?.['required']">⚠ Email is required</div>
    <div *ngIf="emailField.errors?.['pattern']">⚠ Invalid email format</div>
  </div>

  <button [disabled]="myForm.invalid">Submit</button>
</form>
```

Output:

Empty field (touched): ⚠ Email is required
Invalid format: ⚠ Invalid email format
Valid email: Submit button enabled ✓

---

4. Displaying Error Messages

4.1 Basic Error Display

Show error messages only when appropriate - typically when the field is touched and invalid.

Template:

```
<form #myForm="ngForm">
  <div>
    <label>Username:</label>
    <input
      type="text"
      name="username"
      [(ngModel)]="username"
      #usernameField="ngModel"
      required
      minlength="3">

    <!-- Simple: Show if touched and invalid -->
    <div *ngIf="usernameField.touched && usernameField.invalid"
         class="error">
      Invalid username!
    </div>
  </div>
</form>
```

4.2 Specific Error Messages

Display different messages for different validation errors.

Template:

```
<input
  name="username"
  [(ngModel)]="username"
  #field="ngModel"
  required
  minlength="3"
  pattern="^[a-zA-Z0-9_]+$">

<div *ngIf="field.invalid && field.touched" style="color: red;">
  <div *ngIf="field.errors?.['required']">❌ Username is required</div>
  <div *ngIf="field.errors?.['minlength']">❌ Minimum 3 characters</div>
  <div *ngIf="field.errors?.['pattern']">❌ Only letters, numbers, underscores</div>
</div>
```

Output:

```
Input: "" → ❌ Username is required
Input: "ab" → ❌ Minimum 3 characters
Input: "abc@" → ❌ Only letters, numbers, underscores
Input: "john_123" → ✓ Valid
```

4.3 Error Display Patterns

Pattern 1: Show on Touched

```
<div *ngIf="field.touched && field.invalid">Error</div>
```

Shows error only after user interacts with field.

Pattern 2: Show on Dirty or Touched

```
<div *ngIf="(field.dirty || field.touched) && field.invalid">Error</div>
```

Shows error if user typed or left the field.

Pattern 3: Show on Submit Attempt

```
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <!-- Fields here -->
  <div *ngIf="myForm.submitted && field.invalid">Error</div>
</form>
```

Shows error only after user tries to submit.

4.4 Visual Styling for Errors

Component Styles:

```
.form-control.ng-invalid.ng-touched {
  border-color: #dc3545;
}

.form-control.ng-valid.ng-touched {
  border-color: #28a745;
}
```

> **CSS Classes:** Angular automatically adds CSS classes: `ng-valid`, `ng-invalid`, `ng-touched`, `ng-untouched`, `ng-dirty`, `ng-pristine` for styling.

---

5. Custom Validators

5.1 Why Custom Validators?

Built-in validators cover common scenarios, but sometimes you need custom validation logic like:

- Password must not contain username
- Confirm password must match password
- Username must not be already taken (though this requires async validation)
- Special business rules

5.2 Creating a Custom Directive Validator

Custom validators in template-driven forms are created as directives.

Step 1: Create the Validator Directive

```
import { Directive, Input } from '@angular/core';
import { NG_VALIDATORS, Validator, AbstractControl, ValidationErrors } from '@angular/forms';

@Directive({
  selector: '[appNoSpecialChars]',
  providers: [{
    provide: NG_VALIDATORS,
    useExisting: NoSpecialCharsValidatorDirective,
    multi: true
  }]
})
export class NoSpecialCharsValidatorDirective implements Validator {

  validate(control: AbstractControl): ValidationErrors | null {
    const value = control.value;

    // If empty, let 'required' validator handle it
    if (!value) {
      return null;
    }

    // Check if value contains only letters, numbers, and spaces
    const hasSpecialChars = /[^a-zA-Z0-9 ]/.test(value);

    if (hasSpecialChars) {
      return { 'noSpecialChars': true };
    }

    return null;
  }
}
```

Step 2: Register in Module

```
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule } from '@angular/forms';
```

```
import { AppComponent } from './app.component';
import { NoSpecialCharsValidatorDirective } from './validators/no-special-chars.directive';

@NgModule({
  declarations: [
    AppComponent,
    NoSpecialCharsValidatorDirective
  ],
  imports: [
    BrowserModule,
    FormsModule
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Step 3: Use in Template

```
<form #myForm="ngForm">
  <div>
    <label>Display Name:</label>
    <input
      type="text"
      name="displayName"
      [(ngModel)]="displayName"
      #nameField="ngModel"
      required
      appNoSpecialChars>

    <div *ngIf="nameField.invalid && nameField.touched" style="color: red;">
      <div *ngIf="nameField.errors?.['required']">
        Display name is required
      </div>
      <div *ngIf="nameField.errors?.['noSpecialChars']">
        Display name cannot contain special characters
      </div>
    </div>
  </div>
</form>
```

```
Output:

Validation results:

Input: "John Doe" → Valid ✓
Input: "John123" → Valid ✓
Input: "John@Doe" → Invalid ❌ (Display name cannot contain special characters)
Input: "User#123" → Invalid ❌ (Display name cannot contain special characters)
```

5.3 Parameterized Custom Validator

Create validators that accept parameters.

Validator Directive:

```
@Directive({
  selector: '[appForbiddenWord]',
  providers: [{ provide: NG_VALIDATORS, useExisting: ForbiddenWordValidatorDirective, multi:
true }]
})
export class ForbiddenWordValidatorDirective implements Validator {
  @Input('appForbiddenWord') forbiddenWord: string = '';

  validate(control: AbstractControl): ValidationErrors | null {
    if (!control.value || !this.forbiddenWord) return null;

    const isForbidden = control.value.toLowerCase().includes(this.forbiddenWord.toLowerCase());
    return isForbidden ? { 'forbiddenWord': true } : null;
  }
}
```

Usage:

```
<input
  name="username"
  [(ngModel)]="username"
  #field="ngModel"
  [appForbiddenWord]="'admin'">
```

```
<div *ngIf="field.errors?.['forbiddenWord']" style="color: red;">
  Username cannot contain "admin"
</div>
```

Output:

```
Input: "john" → Valid ✓
Input: "administrator" → Invalid ❌
```

5.4 Confirm Password Validator

A practical example comparing two fields.

Validator Directive:

```
import { Directive, Input } from '@angular/core';
import { NG_VALIDATORS, Validator, AbstractControl, ValidationErrors } from '@angular/forms';

@Directive({
  selector: '[appConfirmPassword]',
  providers: [{
    provide: NG_VALIDATORS,
    useExisting: ConfirmPasswordValidatorDirective,
    multi: true
  }]
})
export class ConfirmPasswordValidatorDirective implements Validator {

  @Input('appConfirmPassword') passwordToCompare: string = '';

  validate(control: AbstractControl): ValidationErrors | null {
    const confirmPassword = control.value;

    if (!confirmPassword || !this.passwordToCompare) {
      return null;
    }

    if (confirmPassword !== this.passwordToCompare) {
      return { 'confirmPassword': true };
    }
```

```
    return null;
  }
}
```

Component:

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-password-form',
  templateUrl: './password-form.component.html'
})
export class PasswordFormComponent {
  password: string = '';
  confirmPassword: string = '';
}
```

Template:

```
<form #myForm="ngForm">
  <div>
    <label>Password:</label>
    <input
      type="password"
      name="password"
      [(ngModel)]="password"
      #passwordField="ngModel"
      required
      minlength="6">

    <div *ngIf="passwordField.invalid && passwordField.touched" style="color: red;">
      <div *ngIf="passwordField.errors?.['required']">
        Password is required
      </div>
      <div *ngIf="passwordField.errors?.['minlength']">
        Password must be at least 6 characters
      </div>
    </div>
  </div>

  <div>
    <label>Confirm Password:</label>
    <input
```

```
      type="password"
      name="confirmPassword"
      [(ngModel)]="confirmPassword"
      #confirmField="ngModel"
      required
      [appConfirmPassword]="password">

    <div *ngIf="confirmField.invalid && confirmField.touched" style="color: red;">
      <div *ngIf="confirmField.errors?.['required']">
        Please confirm your password
      </div>
      <div *ngIf="confirmField.errors?.['confirmPassword']">
        Passwords do not match!
      </div>
    </div>
  </div>

  <button type="submit" [disabled]="myForm.invalid">Submit</button>
</form>
```

Output:

Password confirmation flow:

Step 1:
Password: "pass123"
Confirm: "" ← Please confirm your password

Step 2:
Password: "pass123"
Confirm: "pass12" ← Passwords do not match!

Step 3:
Password: "pass123"
Confirm: "pass123" ← Valid ✓
[Submit Button] Enabled

6. Validation Patterns

6.1 Validation Summary

Display all validation errors in one place.

Template:

```
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <!-- Form fields here -->
  <div>
    <input type="text" name="username" [(ngModel)]="username"
           #usernameField="ngModel" required minlength="3">
  </div>

  <div>
    <input type="email" name="email" [(ngModel)]="email"
           #emailField="ngModel" required>
  </div>

  <!-- Validation Summary -->
  <div *ngIf="myForm.submitted && myForm.invalid"
       style="background: #ffebee; padding: 15px; border-left: 4px solid #f44336; margin: 20px
0;">
    <h4 style="margin-top: 0; color: #c62828;">⚠ Please fix the following errors:</h4>
    <ul style="margin-bottom: 0;">
      <li *ngIf="usernameField.invalid">
        Username:
        <span *ngIf="usernameField.errors?.['required']">Required</span>
        <span *ngIf="usernameField.errors?.['minlength']">Minimum 3 characters</span>
      </li>
      <li *ngIf="emailField.invalid">
        Email:
        <span *ngIf="emailField.errors?.['required']">Required</span>
      </li>
    </ul>
  </div>

  <button type="submit">Submit</button>
</form>
```

Output:

When submit clicked with invalid form:

```
┌──────────────────────────────────────┐
│ ⚠ Please fix the following errors: │
│ │
│ • Username: Minimum 3 characters │
│ • Email: Required │
└──────────────────────────────────────┘
```

6.2 Real-time Validation Feedback

Template:

```
<input
  type="password"
  name="password"
  [(ngModel)]="password"
  minlength="8">

<!-- Requirements checklist -->
<div style="font-size: 12px;">
  <div [style.color]="password.length >= 8 ? 'green' : 'gray'">
    {{ password.length >= 8 ? '✓' : 'o' }} At least 8 characters
  </div>
  <div [style.color]="/[A-Z]/.test(password) ? 'green' : 'gray'">
    {{ /[A-Z]/.test(password) ? '✓' : 'o' }} Uppercase letter
  </div>
  <div [style.color]="/[0-9]/.test(password) ? 'green' : 'gray'">
    {{ /[0-9]/.test(password) ? '✓' : 'o' }} Number
  </div>
</div>
```

Output:

Input "pass":

```
o At least 8 characters
o Uppercase letter
```

```
○ Number

Input "Pass123!":
✓ At least 8 characters
✓ Uppercase letter
✓ Number
```

6.3 Disable Submit Until Valid

Common pattern to prevent invalid form submission.

```
<form #myForm="ngForm" (ngSubmit)="onSubmit(myForm)">
  <!-- Form fields -->

  <button type="submit" [disabled]="myForm.invalid">
    Submit
  </button>

  <!-- Or with better UX -->
  <button type="submit"
          [disabled]="myForm.invalid"
          [style.opacity]="myForm.invalid ? 0.5 : 1"
          [style.cursor]="myForm.invalid ? 'not-allowed' : 'pointer'">
    {{ myForm.invalid ? 'Please fix errors' : 'Submit' }}
  </button>
</form>
```

---

7. Summary

7.1 Key Concepts Learned

Concept	Description
Form States	valid, invalid, pristine, dirty, touched, untouched
Built-in Validators	required, minlength, maxlength, pattern, min, max

Concept	Description
HTML5 vs Angular	Use `novalidate` for Angular-only validation
Error Display	Show errors conditionally based on field state
Custom Validators	Create directive validators for custom logic
CSS Classes	Angular adds ng-valid, ng-invalid, etc. for styling

7.2 Validation Checklist

- ✓ Import FormsModule
- ✓ Add `novalidate` to form
- ✓ Use template reference variables (`#field="ngModel"`)
- ✓ Apply validators to inputs
- ✓ Check field states before showing errors
- ✓ Display specific error messages
- ✓ Style invalid fields with CSS
- ✓ Disable submit button when form is invalid

7.3 Common Validation Patterns

```
<!-- Pattern 1: Basic validation -->
<input name="field" [(ngModel)]="value" #field="ngModel" required>
<div *ngIf="field.invalid && field.touched">Error</div>

<!-- Pattern 2: Specific errors -->
<div *ngIf="field.errors?.['required']">Required</div>
<div *ngIf="field.errors?.['minlength']">Too short</div>

<!-- Pattern 3: Disable submit -->
<button [disabled]="myForm.invalid">Submit</button>

<!-- Pattern 4: Validation summary -->
```

```
<div *ngIf="myForm.submitted && myForm.invalid">
  Fix errors before submitting
</div>
```

7.4 Best Practices

1. **Show errors at the right time:** Use `touched` or `dirty` to avoid overwhelming users
2. **Provide clear error messages:** Tell users exactly what's wrong and how to fix it
3. **Use visual feedback:** Color, icons, and borders help users understand validation state
4. **Disable invalid submissions:** Prevent form submission when invalid
5. **Keep validators simple:** Complex validation logic should be in the component, not directives
6. **Test validation thoroughly:** Try edge cases like empty strings, special characters, etc.

7.5 Next Steps

In Module 9.3, you will learn:

- Prepopulating form values
- One-way data binding in forms
- Grouping form controls with ngModelGroup
- Form submission and reset handling
- Working with complex form structures

Additional Resources

- **Angular Forms Guide:** https://angular.io/guide/forms-overview
- **Built-in Validators:** https://angular.io/api/forms/Validators
- **Custom Validators:** https://angular.io/guide/form-validation#custom-validators
- **NgModel API:** https://angular.io/api/forms/NgModel
- **ValidationErrors:** https://angular.io/api/forms/ValidationErrors

---

> **Pro Tip:** Validation is about user experience. Show errors at the right time, provide clear messages, and help users succeed. Don't just block them - guide them!